QLSTM: Gradients & Training Mechanics

Reference Sheet for Presentation

Gautam Gupta — 2023220

1. What is Trainable?

The QLSTM has **125 total trainable parameters** (with $n_q = 2$, $L = 1$, hidden$= 4$), divided into three categories. Every one of these is registered with PyTorch via `nn.Parameter` or `nn.Linear`, so they all show up in `model.parameters()` and are updated by Adam in a single step.

1.1. (A) Classical pre-nets — one per VQC

```
self.pre_net = nn.Linear(input_size, n_qubits)
```

For each gate VQC: $W \in \mathbb{R}^{n_q \times (\text{input}+\text{hidden})}$, $b \in \mathbb{R}^{n_q}$.

With input $= 1$, hidden $= 4$, $n_q = 2$: weights $5{\times}2 = 10$, bias 2, $\Rightarrow$ **12 params per VQC**.

Four VQCs $\Rightarrow$ **48 classical pre-net params total.**

1.2. (B) Quantum parameters (the actual circuit angles)

```
self.q_params = nn.Parameter(torch.randn(n_layers, n_qubits, 3) * 0.1)
```

Shape $(L, n_q, 3)$ — three angles (α, β, γ) per qubit per layer, fed into the `Rot` gate inside `StronglyEntanglingLayers`:

$$\text{Rot}(\alpha, \beta, \gamma) = R_Z(\gamma)\, R_Y(\beta)\, R_Z(\alpha)$$

With $L = 1, n_q = 2$: $1 \cdot 2 \cdot 3 = 6$ angles per VQC, four VQCs $\Rightarrow$ **24 quantum angles total.**

These are the parameters that suffer from barren plateaus.

1.3. (C) Classical post-nets + final readout

```
self.post_net = nn.Linear(n_qubits, output_size)   # per VQC
self.fc_out   = nn.Linear(hidden_size, output_size) # once at top
```

Four post-nets at $4 \cdot (2 \cdot 4 + 4) = 48$ params,[1] plus `fc_out` $= 5$ params.

1.4. Parameter budget summary

$$\underbrace{48}_{\text{4 pre-nets}} + \underbrace{24}_{\text{quantum angles}} + \underbrace{48}_{\text{4 post-nets}} + \underbrace{5}_{\text{fc\_out}} = \boxed{125 \text{ params}}$$

Only 19% of the model is quantum. The rest is classical glue.

[1] Pre/post-net output dim equals `hidden_size`=4 in our config.

2. The Quantum Circuit: Forward Pass Equations

For one VQC call on input vector $x \in \mathbb{R}^{n_q}$ (already pre-net + tanh'd):

2.1. Step 1 — Angle embedding

$$|\psi_0\rangle = \bigotimes_{i=0}^{n_q-1} R_Y(x_i)\,|0\rangle \quad \text{where} \quad R_Y(\theta) = \begin{pmatrix} \cos(\theta/2) & -\sin(\theta/2) \\ \sin(\theta/2) & \cos(\theta/2) \end{pmatrix}$$

2.2. Step 2 — Strongly entangling layers

For each layer $\ell = 1, \ldots, L$:

$$U_\ell(\theta_\ell) = U_{\text{CNOT-ring}} \cdot \prod_{i=0}^{n_q-1} \text{Rot}(\alpha_{\ell,i}, \beta_{\ell,i}, \gamma_{\ell,i})_i$$

Full circuit unitary:

$$U(\theta) = U_L(\theta_L) \cdots U_2(\theta_2)\, U_1(\theta_1)$$

Final state:

$$|\psi(\theta, x)\rangle = U(\theta)\,|\psi_0(x)\rangle$$

2.3. Step 3 — Measurement

$$z_i(\theta, x) = \langle\psi(\theta, x)|\, Z_i\, |\psi(\theta, x)\rangle \in [-1, 1], \qquad i = 0, \ldots, n_q - 1$$

The output of the VQC is the vector $\mathbf{z}(\theta, x) = (z_0, \ldots, z_{n_q-1})^\top$, which then feeds into `post_net`.

What is z_i, concretely? After the circuit produces the final state $|\psi(\theta, x)\rangle$, we measure the Pauli-Z observable on each qubit. The number z_i is the *expectation value* of that measurement on qubit i, where Z_i means "Pauli-Z acting on qubit i, identity on every other qubit":

$$Z_i \;=\; I \otimes I \otimes \cdots \otimes \underbrace{Z}_{\text{position } i} \otimes \cdots \otimes I$$

Pauli-Z has eigenvalues $+1$ on $|0\rangle$ and -1 on $|1\rangle$, so z_i has a clean probabilistic interpretation:

$$z_i \;=\; P(\text{qubit } i = 0) \;-\; P(\text{qubit } i = 1) \;\in\; [-1, +1]$$

It is a single real number summarising "is qubit i more likely to be 0 or 1, and by how much."

What the VQC actually outputs. A VQC with n_q qubits returns n_q such numbers — one expectation value per qubit:

$$\mathbf{z}(\theta, x) \;=\; \big(z_0,\; z_1,\; \ldots,\; z_{n_q-1}\big) \;\in\; [-1, 1]^{n_q}$$

In code (`src/qlstm.py:41`):

```
return [qml.expval(qml.PauliZ(i)) for i in range(n_qubits)]
```

That list comprehension is literally producing $[z_0, z_1, \ldots, z_{n_q-1}]$.

Why these are what we differentiate. The vector $\mathbf{z}$ is the *only* thing the quantum circuit hands back to the classical world — it is the input to `post_net`. Everything downstream (post-net → sigmoid → cell update → loss) is classical. So the quantum/classical boundary lives at $\mathbf{z}$, and PennyLane's job is exactly to compute $\partial z_i/\partial\theta_k$ at that boundary. Once those numbers exist, PyTorch's autograd chains them through to $\partial L/\partial\theta_k$.

Concrete tiny example. Suppose $n_q = 2$ and the final state happens to be the Bell state

$$|\psi\rangle = \tfrac{1}{\sqrt{2}}|00\rangle + \tfrac{1}{\sqrt{2}}|11\rangle.$$

Then

$$z_0 = \langle\psi|Z_0|\psi\rangle = \tfrac{1}{2}(+1) + \tfrac{1}{2}(-1) = 0, \qquad z_1 = \langle\psi|Z_1|\psi\rangle = 0.$$

So the VQC outputs $\mathbf{z} = (0, 0)$ — just two real numbers, which then go into the classical `post_net`.

Summary.

- $|\psi\rangle$ = the full 2^{n_q}-dimensional quantum state, lives inside the simulator.
- z_i = a single real number in $[-1, 1]$, the expectation of Z on qubit i.
- $\mathbf{z}$ = the n_q-vector of these numbers, the actual classical output of the VQC.
- Gradients $\partial z_i/\partial\theta_k$ are what PennyLane computes; everything else is autograd.

2.4. State evolution — what does the qubit register actually look like?

Starting from the all-zero register $|0\rangle^{\otimes n_q}$, the state evolves through three stages:

(a) After angle embedding — apply $R_Y(x_i)$ on each qubit:

$$|\psi_0(x)\rangle \;=\; \bigotimes_{i=0}^{n_q-1} R_Y(x_i)\,|0\rangle \;=\; \bigotimes_{i=0}^{n_q-1} \Big(\cos\tfrac{x_i}{2}\,|0\rangle + \sin\tfrac{x_i}{2}\,|1\rangle\Big)$$

This is still a *product state*: no entanglement yet, just data loaded into single-qubit amplitudes.

(b) After one strongly-entangling layer — per-qubit `Rot` followed by a CNOT ring:

$$|\psi_1(\theta_1, x)\rangle \;=\; U_{\text{CNOT-ring}} \Big[\bigotimes_{i=0}^{n_q-1} \mathrm{Rot}(\alpha_{1,i}, \beta_{1,i}, \gamma_{1,i})\Big]\,|\psi_0(x)\rangle$$

where $U_{\text{CNOT-ring}} = \mathrm{CNOT}_{n_q-1,0} \cdots \mathrm{CNOT}_{1,2}\,\mathrm{CNOT}_{0,1}$. After this layer the state is generally *entangled* and no longer factorises across qubits.

(c) After all L layers — the final state going into the measurement step:

$$\boxed{|\psi(\theta, x)\rangle \;=\; \Big[\prod_{\ell=1}^{L} U_{\text{CNOT-ring}} \bigotimes_{i=0}^{n_q-1} \mathrm{Rot}(\alpha_{\ell,i}, \beta_{\ell,i}, \gamma_{\ell,i})\Big] \bigotimes_{i=0}^{n_q-1} R_Y(x_i)\,|0\rangle}$$

This is an entangled n_q-qubit state living in the full Hilbert space $\mathbb{C}^{2^{n_q}}$, jointly parameterised by the data x and trainable angles θ. It cannot in general be written as a tensor product of single-qubit states — this is precisely what gives the VQC its expressive power, and also what causes barren plateaus when n_q grows large (the state behaves more and more like a Haar-random vector in $\mathbb{C}^{2^{n_q}}$).

The measurement step in §2.3 then collapses this 2^{n_q}-dimensional state down to n_q classical numbers $z_i = \langle\psi|Z_i|\psi\rangle$, which are then handed to the classical `post_net`.

3. How Gradients are Computed

Three different gradient mechanisms run in different parts of the graph. The chain rule glues them together.

3.1. Mechanism 1 — Standard PyTorch autograd (classical parts)

For `pre_net`, `post_net`, `fc_out`, `tanh`, `sigmoid`, the cell-state update — everything is just tensor ops. PyTorch builds the computation graph during the forward pass and reverse-walks it during `loss.backward()`.

3.2. Mechanism 2 — Quantum gradients via `diff_method="backprop"`

When autograd hits `self.circuit(x[i], self.q_params)`, PennyLane takes over. With `diff_method="backprop"` on the `default.qubit` simulator, PennyLane internally represents the circuit as a sequence of unitary matrix multiplications on a state vector, and backpropagates symbolically.

It returns gradients with respect to *both*:

$$\frac{\partial z_i}{\partial \theta} \quad \text{and} \quad \frac{\partial z_i}{\partial x}$$

The second one is what allows the classical pre-net to receive gradient signal.

What is the "objective" here, exactly? The thing being differentiated at this stage is just the measurement output

$$z_i(\theta, x) = \langle \psi(\theta, x) | \, Z_i \, | \psi(\theta, x) \rangle$$

treated as a function of θ (and of x). We are *not* differentiating the loss yet — the loss-gradient is built up later by the chain rule, using PyTorch autograd to stitch together all the classical pieces. PennyLane's job is just to deliver the partial $\partial z_i / \partial \theta_k$ at the quantum boundary.

How `backprop` actually computes the gradient. On a simulator, the quantum state is a complex vector $|\psi\rangle \in \mathbb{C}^{2^{n_q}}$ and every gate is a matrix multiplication $|\psi\rangle \mapsto U_k |\psi\rangle$. So the entire circuit is just a sequence of differentiable tensor operations:

$$|\psi(\theta, x)\rangle = U_L(\theta_L) \cdots U_2(\theta_2) \, U_1(\theta_1) \, |\psi_0(x)\rangle, \qquad z_i = \langle \psi | \, Z_i \, | \psi \rangle$$

PennyLane registers each $U_\ell(\theta_\ell)$ inside PyTorch's autograd graph as a `torch.Tensor` matrix-multiply with `requires_grad=True`. When `loss.backward()` is called, standard *reverse-mode automatic differentiation* flows through:

1. the final z_i, giving $\partial z_i / \partial |\psi\rangle$;
2. the chain $U_L \cdots U_1$, one matrix at a time;
3. at each $U_\ell(\theta_\ell)$, autograd uses the closed-form derivative (since $U = e^{-i\theta P/2}$ has a known symbolic derivative);
4. out comes $\partial z_i / \partial \theta_k$ for *all* angles in one reverse sweep.

Why it is exact. There is no finite-difference approximation anywhere. Every step is an analytic tensor operation, just like differentiating $\sin\theta$ symbolically rather than numerically. The same exactness as classical `nn.Linear` backprop — only the operations happen to be complex matrix multiplications instead of real ones.

Why we use it in this project. On the simulator, `backprop` is *much* faster than parameter-shift (one reverse sweep vs. $2P$ extra forward passes), and gives mathematically identical gradients. Since we are studying trainability and barren plateaus, not benchmarking on real hardware, the cheaper method suffices. See `src/qlstm.py:34`.

3.3. Mechanism 3 — Parameter-shift rule (real hardware)

If you change `diff_method="backprop"` to `diff_method="parameter-shift"`, PennyLane computes each quantum gradient via the famous shift identity:

$$\boxed{\frac{\partial z_i}{\partial \theta_k} \;=\; \frac{1}{2}\Big[\, z_i\big(\theta + \tfrac{\pi}{2}\,\mathbf{e}_k\big) \;-\; z_i\big(\theta - \tfrac{\pi}{2}\,\mathbf{e}_k\big)\,\Big]}$$

What this rule says, in words. For *each* quantum angle θ_k that needs a gradient, we run the *entire circuit twice*:

1. Once with θ_k replaced by $\theta_k + \pi/2$, all other angles fixed — get measurement $z^{(+)}$.
2. Once with θ_k replaced by $\theta_k - \pi/2$, all other angles fixed — get measurement $z^{(-)}$.

Then $\partial z_i/\partial\theta_k = \frac{1}{2}(z^{(+)} - z^{(-)})$. That is the gradient. No finite-difference approximation, no Taylor expansion — it is an algebraic identity, not a numerical estimate.

Why it is exact (one-line proof). For a gate of the form $U(\theta) = e^{-i\theta P/2}$ where P is a Pauli (so $P^2 = I$):

$$U(\theta) = \cos(\theta/2)\, I \;-\; i\sin(\theta/2)\, P$$

Compute $z(\theta) = \langle\psi(\theta)|\, O\, |\psi(\theta)\rangle$ and take $\partial/\partial\theta$. The result, after using $\sin' = \cos$ and the addition formulas, is *exactly*

$$\frac{\partial z}{\partial \theta} = \tfrac{1}{2}\big[z(\theta + \pi/2) - z(\theta - \pi/2)\big].$$

The trick is that a $\pm\pi/2$ shift swaps $\sin \leftrightarrow \cos$, so the derivative reduces to a finite combination of shifted evaluations.

Cost. Two extra full circuit evaluations *per quantum parameter*. With 24 quantum angles in our QLSTM, that is 48 extra forward passes for one backward step. Multiply by sequence length 10 and 4 gates per cell and you understand why running QLSTM on real hardware is brutally slow.

Chain rule still applies. Parameter-shift only computes $\partial z_i/\partial\theta_k$. To get the loss gradient, the standard chain rule kicks in:

$$\frac{\partial L}{\partial \theta_k} \;=\; \underbrace{\frac{\partial L}{\partial z_i}}_{\text{from PyTorch autograd}} \cdot \underbrace{\frac{\partial z_i}{\partial \theta_k}}_{\text{parameter-shift rule}}$$

PennyLane handles both pieces and hands the final scalar gradient to PyTorch.

3.4. `backprop` vs. parameter-shift — side-by-side

	`backprop`	parameter-shift
Where it works	Simulator only	Simulator + real hardware
Cost / backward	1 reverse-mode sweep	$2P$ extra circuit runs
Memory	$O(2^{n_q})$ (state vector stored)	$O(1)$ (only run circuits)
Exactness	Exact (symbolic differentiation)	Exact (algebraic identity)
Why exact	Direct chain rule on matrix ops	$\sin \leftrightarrow \cos$ at $\pm\pi/2$
Real-hardware compatible	No	Yes

One sentence each:

- `backprop` is "I have full access to the simulator's state vector, so I'll just chain-rule through the matrix multiplications."
- parameter-shift is "I have only a black-box quantum device that runs circuits and measures; I'll exploit the algebraic identity $\sin \leftrightarrow \cos$ to recover exact gradients with two extra runs per parameter."

Both produce the same numerical gradient. We use `backprop` because we are not on hardware.

4. The Full Chain Rule (End-to-End)

For a single training sample, sequence length T, the gradient of the loss L with respect to one quantum angle θ_k inside `vqc_forget` is:

$$\frac{\partial L}{\partial \theta_k} = \sum_{t=1}^{T} \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\text{loss head}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial h_T}}_{\text{fc\_out}} \cdot \underbrace{\frac{\partial h_T}{\partial h_t}}_{\text{BPTT}} \cdot \underbrace{\frac{\partial h_t}{\partial f_t}}_{\text{sigmoid + cell update}} \cdot \underbrace{\frac{\partial f_t}{\partial \mathbf{z}_t}}_{\text{post\_net}} \cdot \underbrace{\frac{\partial \mathbf{z}_t}{\partial \theta_k}}_{\text{PennyLane}}$$

Reading the chain (right to left):

1. `loss.backward()` starts at the loss.
2. Autograd flows through `fc_out`, the unrolled time loop (BPTT), the cell-state update, the sigmoid, and the post-net — all classical.
3. At the boundary $\partial \mathbf{z}_t / \partial \theta_k$, **PennyLane handles the quantum part** via `diff_method`.
4. Gradient also flows back through $\partial \mathbf{z}_t / \partial x$ into the pre-net.

4.1. BPTT — backprop through time

Because `QLSTM.forward` runs a Python loop `for t in range(seq_len)` (line 139), the cell is unrolled T times in the computation graph. The gradient $\partial h_T / \partial h_t$ recursively expands as:

$$\frac{\partial h_T}{\partial h_t} = \prod_{\tau=t+1}^{T} \frac{\partial h_\tau}{\partial h_{\tau-1}}$$

With $T = 10$ and 4 gates, each backward pass executes the quantum-gradient call $10 \times 4 = 40$ **times per sample**.

5. The Barren Plateau Problem (mathematically)

5.1. The killer result — McClean et al. 2018

For a randomly-initialised parameterised quantum circuit with n qubits drawn from a unitary 2-design:

$$\mathbb{E}_\theta\left[\frac{\partial L}{\partial \theta_k}\right] = 0 \quad \text{and} \quad \boxed{\mathrm{Var}_\theta\left[\frac{\partial L}{\partial \theta_k}\right] \in O\left(\frac{1}{2^n}\right)}$$

The mean is zero (so gradients are signed noise), and the variance *decays exponentially* with qubit count.

5.2. What this means for Adam

Adam's update rule:

$$\theta \leftarrow \theta - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \varepsilon} \qquad \hat{m}_t \approx \mathbb{E}[g_t], \;\; \hat{v}_t \approx \mathbb{E}[g_t^2]$$

If $\mathrm{Var}[g] \to 0$, then $\hat{v}_t \to 0$ and gradients are dominated by numerical noise. Adam moves the quantum angles by negligible amounts $\Rightarrow$ they stay frozen near random init $\Rightarrow$ training fails.

Crucially, the classical parameters keep updating — they're not subject to BP. So you'd see the loss drop a little (from classical fits) and then stall, with the quantum block never having contributed.

5.3. Empirical measurement (from `src/bp_full_grid.py`)

For each (n, L) pair:

1. Sample $\theta \sim U[0, 2\pi]^P$ and a random input.
2. Compute one forward + backward pass to get $\nabla_\theta L \in \mathbb{R}^P$.
3. Repeat 200 times.
4. Compute per-parameter variance, then mean across parameters:

$$\widehat{\mathrm{Var}}[\nabla L] = \frac{1}{P}\sum_{k=1}^{P} \underbrace{\frac{1}{N-1}\sum_{s=1}^{N}\left(g_k^{(s)} - \bar{g}_k\right)^2}_{\text{sample variance for parameter } k}$$

Why mean across parameters and not just θ_0: if θ_0 controls a Z-rotation immediately before a Z measurement, then $[Z, Z] = 0$ gives a structural zero gradient that has nothing to do with BP. Averaging avoids this artefact.

6. The Optimiser Step

Once gradients exist, the optimiser doesn't care where they came from:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.02)
loss.backward()      # populates .grad on all 125 Parameters
optimizer.step()     # one Adam update for all of them
```

`model.parameters()` returns all classical *and* quantum leaf tensors indistinguishably. Adam applies the same update rule to every one. There is no special "quantum optimiser."

7. Sanity-check Snippet (Live Demo)

Run this during the talk to show all gradients flow:

```
model = QLSTM(input_size=1, hidden_size=4, output_size=1,
              n_qubits=2, n_layers=1)
loss = model(torch.randn(2, 10, 1)).sum()
loss.backward()

for name, p in model.named_parameters():
    print(f"{name:50s} shape={tuple(p.shape)}  "
          f"grad_norm={p.grad.norm().item():.4f}")
```

You will see:

- All 125 parameters have non-zero gradients ⇒ training works end-to-end.
- `q_params` gradients are smaller than classical ones ⇒ foreshadows BP.
- Each VQC's pre-net, q_params, and post-net appear separately in the list.

8. One-Slide Summary

Component	Gradient by	Count
4 pre-nets	PyTorch autograd	48 params
4 VQC angle sets	PennyLane (`backprop` or shift)	24 params
4 post-nets	PyTorch autograd	48 params
1 fc_out	PyTorch autograd	5 params
Total	all glued by chain rule	**125 params**

The whole hybrid network trains with one `loss.backward()` + one `optimizer.step()`. PennyLane is the only "unusual" piece, and it just provides the gradient $\partial \mathbf{z}/\partial\theta$ at the quantum boundary. Everything else is classical deep learning.

9. Two-Stage Least-Squares Mitigation — Simple Explanation

9.1. The setup in one picture

The QLSTM has this shape:

$$\text{input } X \xrightarrow{\text{QLSTM cell}} h_T \xrightarrow{\texttt{fc\_out}} \hat{y}$$

Everything before `fc_out` is non-linear (and contains the quantum stuff). `fc_out` is just one linear layer:

$$\hat{y} \;=\; \texttt{fc\_out}(h_T) \;=\; w^\top h_T + b$$

Key observation: if we *freeze* the QLSTM cell, then training the model is just *linear regression* from h_T to $\hat{y}$. Linear regression has a closed-form solution. No gradient descent needed.

9.2. What is Φ?

Φ is just the matrix of h_T **values**, one row per training sample.

For our S&P 500 setup:

- Training samples: $N = 80$

- Hidden size: $d = 4$

So for each training sample i, the QLSTM eats the 10-day price sequence and spits out a 4-dimensional vector $h_T^{(i)} \in \mathbb{R}^4$. Stack all 80 of them:

$$\Phi \;=\; \begin{pmatrix} h_T^{(1)\top} \\ h_T^{(2)\top} \\ \vdots \\ h_T^{(80)\top} \end{pmatrix} \;\in\; \mathbb{R}^{80\times 4}$$

In one sentence: Φ is what comes out of the QLSTM, before the final linear layer, for every training sample.

9.3. What is y?

y is the matrix of **true labels** you want to predict. For S&P 500, the label is the next-day price (one number per sample):

$$y \;=\; \begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(80)} \end{pmatrix} \;\in\; \mathbb{R}^{80\times 1}$$

In one sentence: y is just the training labels (true next-day prices), stacked.

9.4. The closed-form solution

We want to find weights $w \in \mathbb{R}^{4\times 1}$ such that

$$\Phi w \;\approx\; y.$$

Schematically:

$$\underbrace{\begin{pmatrix} \cdot & \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot & \cdot \\ & \vdots & & \\ \cdot & \cdot & \cdot & \cdot \end{pmatrix}}_{\Phi\text{: } 80\times 4} \underbrace{\begin{pmatrix} w_1 \\ w_2 \\ w_3 \\ w_4 \end{pmatrix}}_{w\text{: } 4\times 1} \;\approx\; \underbrace{\begin{pmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(80)} \end{pmatrix}}_{y\text{: } 80\times 1}$$

This is just linear regression. Solving the ridge version (with λI for stability):

$$\boxed{w^* \;=\; \left(\Phi^\top \Phi + \lambda I\right)^{-1} \Phi^\top y}$$

That gives the best 4 numbers (w_1, w_2, w_3, w_4). We copy them into `fc_out.weight`. Done with Stage 1.

9.5. Stage 2 — now train normally

After Stage 1, we have a great `fc_out`. Stage 2 just runs normal Adam training on *all* parameters (quantum + classical), exactly like the random-init baseline. The only difference: we start from a better point.

9.6. Why it fails on S&P 500

Here is the simple version of the problem:

Stage 1 fits w to the features that the *starting* (random) quantum parameters happen to produce.

Call those features Φ_0.

Stage 2 changes the quantum parameters.

Once the quantum parameters move, the features change too. Call them Φ_t. So:

$$w^* \text{ was optimal for } \Phi_0, \qquad \text{but now we have } \Phi_t \neq \Phi_0.$$

The carefully-tuned `fc_out` is no longer matched to the features it sees. We have basically broken the warm-start. On S&P 500 (which is non-stationary, so over-fitting is dangerous), this hurts:

Method	**Test MAE**
Random init QLSTM	1.242
Identity-block init QLSTM	1.246
Two-stage LS warm-up	**2.374** (worse!)

9.7. What we did novel — the fix

The diagnosis is: *features drift away from where Stage 1 solved.*

The fix is: **add a penalty that keeps the features from drifting.**

We **save** Φ_0 after Stage 1. Then during Stage 2, at every training step, we compute the current features Φ_t and add a penalty if they have moved:

$$L_{\text{total}}(\theta) = \underbrace{\|\hat{y} - y\|^2}_{\text{usual MSE}} + \lambda_{\text{feat}} \cdot \underbrace{\|\Phi_t - \Phi_0\|^2}_{\text{don't drift!}}$$

In words: "Train normally, but also pay a price every time the QLSTM features stray from where they were in Stage 1."

In code (`src/mitigations.py:train_with_feature_penalty`):

```
def hook(mod, inp, out):
    captured["features"] = inp[0]   # KEEP gradient (no .detach())

handle = model.fc_out.register_forward_hook(hook)

for _ in range(epochs):
    pred       = model(X)                          # forward pass
    data_loss  = loss_fn(pred, y)
    Phi_t      = captured["features"]              # current features
    feat_pen   = ((Phi_t - Phi_0) ** 2).mean()
    total      = data_loss + feature_lambda * feat_pen
    total.backward()                               # gradient flows
    opt.step()                                     # quantum + classical
       updated
```

The *key* line: in Stage 1's hook we used `inp[0].detach()` (we just wanted the numbers). In Stage 2 we do **not** detach — so the penalty's gradient flows back through PennyLane into the quantum angles. Adam then nudges θ_q to keep the features stable.

9.8. Why this is novel

- The original Boabang & Gyamerah paper (2026) does Stage 2 as plain Adam — exactly the recipe that broke for us.
- None of the 5 papers I cite use a feature-stability penalty. It came directly from looking at *my own* failed experiment and asking "what is going wrong?"
- It is implemented and runnable in `src/mitigations.py` (functions `two_stage_ls_novel_warmup` and `train_with_feature_penalty`).

Three behaviours from one knob. The hyper-parameter λ_{feat} controls everything:

- $\lambda_{\text{feat}} = 0 \Rightarrow$ original (broken) two-stage LS.
- $\lambda_{\text{feat}} = \infty \Rightarrow$ features frozen $\Rightarrow$ only `fc_out` can move (pure linear regression on random quantum features).
- λ_{feat} moderate $\Rightarrow$ keeps the warm-start *and* lets the quantum block adapt slowly.

9.9. Side-by-side

Original two-stage LS	**Our novel variant**
Stage 1: solve $w^* = (\Phi_0^\top \Phi_0 + \lambda I)^{-1} \Phi_0^\top y$	Same Stage 1, but *also save* Φ_0
Stage 2: plain Adam on MSE	Stage 2: Adam on MSE$+\lambda_{\text{feat}}\|\Phi_t - \Phi_0\|^2$
Quantum features drift freely	Quantum features pulled back toward Stage-1
Hurts on S&P 500 (MAE 2.374)	Designed to keep the warm-start gain

One-line takeaway. Stage 1 is just ridge regression: Φ = QLSTM features for all training samples, y = true labels, $w^* = (\Phi^\top \Phi + \lambda I)^{-1} \Phi^\top y$. Stage 2 of the original paper breaks the Stage 1 fit because the features drift. Our novel fix saves Φ_0 and adds $\lambda_{\text{feat}} \|\Phi_t - \Phi_0\|^2$ to the loss so they cannot drift.